RACRP — everything, from scratch

Written to be read cold. No background assumed. If you understand this document you can present the project and answer questions about it.

Every number here comes from a file under `artifacts/tables/`. Nothing is rounded up, invented, or borrowed from another paper.

---

1. The problem, from zero

- A normal web server sits running all the time, waiting for requests. You pay for it all the time, even at 4am when nobody is using it.
- **Serverless** is the alternative: the cloud shuts your code down when nobody is using it, and starts it up again when a request arrives. You only pay while it runs. This is why it exists.
- The catch: starting up takes time. Downloading the code, booting the sandbox, initialising the runtime. Hundreds of milliseconds to several seconds.
- That startup delay, paid by a real user waiting, is a **cold start**. It is the single biggest complaint about serverless computing.
- You can avoid a cold start by keeping the container alive while idle. But idle memory costs money.
- **So the whole problem is one decision, repeated forever: after an app goes quiet, how long do you keep it alive before shutting it down?**
- Shut down too early → cold start, user waits. Shut down too late → you paid for memory doing nothing.

The unit we measure cost in

- Cost is **GB-minutes**: one gigabyte of memory held for one minute.
- An app with 1 GB allocated, kept warm for 30 idle minutes, cost 30 GB-minutes.
- All our numbers are "cold starts, at some number of GB-minutes spent".

---

2. What the industry actually does today (our baseline)

- Microsoft published the real policy running in Azure Functions, in a paper at USENIX ATC 2020 (Shahrad et al.). This is the thing we must beat. Not a textbook strawman — the actual deployed system.
- It works per application, like this:

1. Watch the app. Every time it goes quiet and then gets a request, record how long that quiet period was. Call that an **idle gap**.
2. Collect all those gap lengths into a **histogram** — a bar chart of "how often was the gap 1 minute, 2 minutes, 3 minutes, ... ".
3. Read two numbers off that histogram:
   - the **5th percentile** → shut down now, come back after this long ("pre-warm offset")
   - the **99th percentile** → having come back, stay alive this long ("keep-alive length")

- Concretely: if 5% of this app's gaps are under 2 minutes and 99% are under 60 minutes, the policy releases the container, brings it back at minute 2, and holds it until minute 60.
- That is the entire deployed policy. A histogram, and two percentiles read off it.

---

3. THE KEY IDEA THAT MAKES THIS PROJECT MAKE SENSE

This is the part that was never stated clearly, and without it nothing else lands. Read this section twice.

Any policy of this kind is really doing two separate things:

- **Step A — which histogram do you look at?** You have to build a picture of "how long are this app's gaps usually".
- **Step B — how do you turn that histogram into an actual window?** You have a chart; you need two numbers, "come back at X, hold for Y".

These are completely independent. You can change one without touching the other. Nobody in the literature had separated them. Once you do, there are four possible policies, not one:

- **Marginal histogram + percentile window** → this is Microsoft's deployed policy. Our baseline.
- **Conditional histogram + percentile window** → change Step A only. This was our first result. **+26.3%**
- **Marginal histogram + derived window** → change Step B only. This is our headline. **+78.8%**
- **Conditional histogram + derived window** → change both. **+68.6%**, which is *worse* than changing Step B alone.

When I said earlier "two things that don't combine", these are the two things. Changing Step A and changing Step B. Not two settings of one idea — two genuinely different modifications, in two different places.

Everything below explains each step, why we changed it, and what happened.

---

4. The rules we forced on ourselves before measuring anything

These matter more than any result. If an examiner attacks the project, it will be here, and these are the answers.

- **Matched cost, always.** Any policy can cut cold starts by simply holding containers warm longer. That is not cleverness, it is spending more money. So we never compare two policies at one setting each. We sweep every policy across its whole range, draw a curve of cold starts against GB-minutes, and compare only at *equal spend*.
  - Why we take this so seriously: an early version of this project showed a 38% improvement that **completely disappeared** once we matched the budget. It was buying the result with memory.
- **Never train or tune on the test data.** Days 1-8 to learn, days 9-10 to choose settings, days 11-14 to report. Never shuffled — you cannot predict Monday using Thursday.
- **Everything decided causally.** At any minute, the policy may only use gaps that have already *finished* before that minute. There is a unit test that changes the trace after minute *t* and asserts no decision before *t* moves.
- **Every claim gets a control.** A control is a deliberately broken version of our own method. If the broken version scores as well as the real one, the real one wasn't doing anything. We report the control next to the result, always.

---

5. What we built first, and why all of it failed

The project's original plan was retrieval-augmented prediction: let each app learn from other apps by looking up similar situations across the fleet. We built it. It did not work. Four separate failures:

- **Failure 1 — cross-app situation retrieval. Result: nothing.** We embedded each app's recent activity, indexed the whole fleet, retrieved the most similar past situations from other apps, and fed them to a transformer.
  - The control that killed it: we fed the model **random** memories instead of real ones. It scored the same (0.9854 vs 0.9851). And a plain model with no retrieval at all, given the same amount of training, **beat** it (0.9862).
  - Meaning: the "retrieval gain" was just extra model capacity and extra training steps. The retrieved information contributed zero.
- **Failure 2 — borrowing gap distributions from similar apps. Result: worse than baseline.** We also built a cheating version that was allowed to peek at the answer and pick the apps whose true distributions were genuinely closest. **That lost too.** So it is not a matter of finding better neighbours. Applications simply are not like each other.
- **Failure 3 — a neural network predicting cold-start risk. Result: 0.98 accuracy, and useless.** It scored 0.98 average precision, which sounds excellent. Converted into an actual keep-alive policy at matched cost, it was **63-118% worse than a plain histogram**.
  - This is the single most useful lesson in the project: **a high prediction score is not a good decision.** The model was accurate about the 99% of minutes that are easy and irrelevant.
- **Failure 4 — conditioning on richer context.** Time of day, day of week, recent activity shape, nearest-neighbour lookups. All lost.

We report all four in the report and on a slide. They are not embarrassments; they are what makes the eventual result believable.

---

6. The measurement that turned the project around

Instead of guessing what to try next, we stopped and measured how predictable this problem is *at all*, without any model. This is called an **information audit** — it uses mutual information, which measures how much knowing one thing reduces your uncertainty about another.

Measured over 264 applications:

- **88.95% of the next idle gap is irreducible.** No model of any kind, however clever, can predict it from anything a scheduler can observe. It is close to random.
- The rest is spread thinly across contexts. Share of the gap's uncertainty each one explains:
  - previous gap length — **11.0%** (the most of anything)
  - recent 5-minute activity — 8.6%
  - time of day — 7.6%
  - recent 60-minute activity — 5.5%
  - day of week — 3.7%
  - burst level — 0.0%

What this single measurement bought us:

- It explained all four failures at once. We had been building sophisticated machinery to extract signal that mostly does not exist.
- It told us where the little signal that *does* exist lives: in one simple, coarse variable — how long the previous gap was.
- It told us what kind of method to use. When the useful signal is one coarse discrete variable, the right tool is a lookup table with thousands of samples per cell, **not** a learned embedding with 8 neighbours per query. A neural network there is pure variance.

If you say one thing about method selection, say this: we measured before we built, and the measurement chose the method.

---

7. CHANGE A — the histogram remembers what state the app was in (+26.3%)

What is wrong with the deployed histogram

- The deployed policy pools **every gap the app has ever had** into one chart.
- Think about an app that gets bursts of traffic. Inside a burst, gaps are 1-2 minutes. Between bursts, gaps are 3 hours.
- Pool them and you get one chart that is a blend of both — and it is wrong in both situations. Too aggressive during the burst, too cautious between bursts.
- Analogy: describing someone who sleeps 8 hours at night and naps 20 minutes at noon with a single "average sleep" number. Useless for predicting either.
- The technical name for this pooled chart is a **marginal distribution**. It has thrown away the *state* the app was in each time.

What we do instead — this is what "keeping state memory" concretely means

- We keep **several histograms per app instead of one**.
- Which one you use depends on how long the *previous* gap was. We bucket the previous gap into a few coarse ranges (very short, short, medium, long, very long).
- So the app has: "here is my gap distribution *given that the last gap was short*", "here is my gap distribution *given that the last gap was long*", and so on.
- At decision time: look at the gap that just finished, pick the matching histogram, read the window off that one.
- That is it. Same policy shape as Microsoft's — still a histogram, still a percentile — but you look up which histogram based on the current state.

The problem that creates, and the fix

- If you split one chart into five, each chart has a fifth of the data. Some buckets barely have any observations, and a window read off three data points is nonsense.
- Fix: **partial pooling**. Blend each conditional histogram towards the app's overall histogram, weighted by how much evidence the bucket actually has.

  - Bucket with lots of observations → trust the bucket.
  - Bucket with almost nothing → fall back to the overall chart.
  - The blend weight is measured in "equivalent observed gaps" (kappa = 8, i.e. the overall chart counts as 8 observations of evidence).
- Nice property: **crank the blend weight to infinity and you get Microsoft's policy back exactly.** Our method is a strict generalisation of theirs, not a competing heuristic. Their policy is our special case.

Result

- **+26.3% fewer cold starts at matched memory cost** (range +19.0% to +30.7%), 1,999 applications, held-out days.
- The control: replace "previous gap length" with a **random label** that splits the data into exactly the same number of buckets. That scored **-1.4%**, essentially zero. So the gain came from the real information, not from the statistical side-effect of slicing the data more finely.
- Robustness: positive at all 15 hyper-parameter settings we swept (+25.2% to +32.7%), and the values we picked were not the grid's best — which is what you expect if you chose them before looking.

---

8. CHANGE B — derive the window instead of guessing it (+78.8%)

This is the bigger result, and it comes from a question nobody had asked, including us, for most of the project.

The question

- **Why the 99th percentile?**
- There is no argument for it anywhere in the literature. It is a number that stuck.
- And there is a worse problem. If you run a cloud platform and someone asks "which percentile do you want?" — that is not a question you can answer. It is not a business quantity. It is an artefact of the shape of a chart.
- But you *can* answer: **"how much is a cold start worth to me, in memory?"** That is a price. Every platform operator knows it.

What we do instead

- Write down what a warm window actually costs you.
- If you keep the container warm over some window, you pay:
  - **memory** for however much of the idle gap that window covers, plus
  - **a cold start** if the request arrives outside your window.
- In symbols: `Cost(a, L) = m × (expected minutes held) + K × P(you missed)` where `a` is when you come back, `L` is how long you hold, `m` is the price of a warm minute and `K` is the price of a cold start.
- Then simply **pick the (a, L) that minimises that cost.** No percentile anywhere.
- The knob you sweep is `K/m` — how many GB-minutes a cold start is worth.

A concrete example of why this beats a percentile

- Suppose an app's gaps are mostly 1-10 minutes, but once in a while 200 minutes.
- The 99th percentile is 200, so the deployed policy holds the container for **200 minutes every single time**.

- What does minute 11 through minute 200 actually buy? It catches maybe the last 1% of gaps, and it costs 190 minutes of memory on every gap.
- If a cold start is worth 20 GB-minutes, that trade is terrible. The cost model sees this immediately and stops at about minute 16.
- **A percentile sees two points on a curve. The cost model integrates the whole shape between them.** That is the entire difference.

One thing we got wrong and it is worth telling them

- Taking the derivative of that cost and setting it to zero gives a genuinely beautiful result: *release the container the moment its hazard rate drops below the price ratio*. It is the kind of thing you want on a slide.
- **We implemented it and it was catastrophic.** Reason: setting the derivative to zero finds a *flat point*, not necessarily the *lowest* point. For a perfectly periodic app — say it fires exactly every 20 minutes — the hazard is zero everywhere before minute 20, so the rule stops at minute 1 and cold-starts on **every single invocation**.
- Fix: don't solve the equation, just search all the options and take the actual minimum. Costs the same amount of computation. It is now pinned by a test against brute-force search.
- The elegant identity is still the right way to *understand* the rule. It is just not the algorithm.

Result

- **+78.8% fewer cold starts at matched memory cost** (range +15.6% to +89.0%), 1,999 applications, held-out days, versus the deployed policy.
- Three times the size of Change A.

The fairness objection, answered before it is raised

- Our rule tunes two things (when to come back, how long to hold). Our sweep of the baseline varied only one. **That is a real asymmetry and an examiner will spot it.**
- So we removed it. We swept the baseline over **both** of its percentiles — 6 × 6 = 36 configurations — and took the best cost/benefit frontier any of them could reach. We even allowed that best-of-36 to be chosen while looking at the test period, which is more help than it deserves.
- **We still win by 67.6% median, and we win at every one of the 13 budgets tested.**
- If challenged on fairness, quote 67.6%, not 78.8%. It is the number we constructed against ourselves.

---

9. THE MOST IMPORTANT NUMBER: −427%

- A 78.8% improvement should make anyone suspicious. The obvious sceptical reading is: *"your rule just happens to produce sensible window sizes; any app would benefit; you haven't learned anything from anyone's history."*
- So we ran the **exact same optimiser** — same cost model, same price, same simulator, same accounting — but gave each app **a different app's history** to work from. Only the ownership of the evidence changed.
- Result: **−427.5%.** Roughly five times *worse* than the baseline it had been beating.
- **Why it fails so badly:** a cost-optimal window computed from the wrong distribution is confidently, precisely wrong. It commits hard to a window that has nothing to do with this app's behaviour. A vague heuristic would degrade gently; an optimiser does not.

- **What it proves:** the method is genuinely reading each application's own past. It is not a lucky guess about window sizes.
- This is the number to say **first**, not last. The improvement is only interesting because the control fails this badly.

---

10. Why Change A and Change B do not combine

- Change A alone: **+26.3%**
- Change B alone: **+78.8%**
- Both together: **+68.6%** — worse than Change B alone, and far more erratic (its range dips to −20.7%, where B alone never drops below +15.6%).
- **The reason, in one sentence:** a percentile is a crude, sturdy number that survives having very little data, whereas a cost-optimal window depends on the *whole shape* of the distribution — especially the rare long gaps in the tail — and Change A chops the data into smaller buckets, which is exactly what ruins your picture of that tail.
- Put differently: **each idea spends the resource the other one needs.** Change A spends data to buy context. Change B needs data to see the shape.
- We report this rather than quietly picking whichever combination looked best. It is a genuine finding about when conditioning is and is not worth it.

---

11. Why some applications improve enormously and others not at all

Expect this question. We measured it rather than guessing.

For Change A (conditioning), we know exactly why

- The whole premise is that the previous gap tells you something about the next one. For some apps it does; for others their gaps are independent — knowing the last one tells you nothing.
- So before running anything, we committed to a falsification test: group apps by how much their consecutive gaps actually depend on each other (measured on *training* data only), then check the gain within each group.
- Measured over 313 applications:
  - **Q1**, gaps carry no information about each other: **–0.6%**
  - **Q2**: −11.6%
  - **Q3**: +8.6%
  - **Q4**, gaps most dependent: **+62.4%**
  - Correlation between "how much information is there" and "how much we gained": **+0.99**
- **This is the strongest evidence in the project that the mechanism is what we say it is.** The method helps exactly where the theory says it must, and vanishes exactly where the theory says it must. A method that helped everywhere equally would be suspicious.
- Note Q2 is *negative*. We report that too.

For Change B (derived window), the reasons an app sees little change

- **Its gaps are all about the same length.** If an app fires every 20 minutes like clockwork, the percentile and the cost-optimal window land in the same place. There is nothing to fix.
- **It has very little history.** With only a handful of observed gaps, neither rule has a shape to work with and both fall back to something similar.
- **Memory is very cheap in that regime.** This is the big one, and it is visible in our own results: the gain falls from about 76% at tight memory budgets to **26.8%** at the loosest one we tested. If memory is nearly free, the right answer is to hold everything warm for a long time, and there is nothing left for an optimiser to recover. Our method matters when memory is scarce enough to be worth allocating carefully — which is the situation a real platform is in.

Why a few applications show enormous negative numbers

This is worth understanding because it looks alarming and it is not what it appears to be. It is a *measurement* artefact on a specific kind of application, not a failure of the method.

- Some applications are almost perfectly regular. Measured example: one whose 1,939 observed gaps take only **seven distinct values**, almost all exactly 5 minutes, with the longest around 10.
- For an application like that, the cost/benefit curve is a **step, not a smooth trade-off.** Either your window is long enough to cover the period, or it is not. There is no useful middle, because there are no gap lengths in between to trade off against each other.
- So our policy's operating points jump. On that application: at one price it holds 5-6 minutes and takes **193 cold starts for 21 GB-minutes**; at the next price it holds 5-12 minutes and takes **1 cold start for 212 GB-minutes**. Nothing exists between those two.
- ATC'20 happens to sit at 191 GB-minutes — right inside that jump. Matched-cost scoring interpolates a straight line between our two points and reads off about 22 cold starts. **But our policy never operates at 191 GB-minutes.** The number describes a point on the curve that does not exist.
- And because the absolute counts are tiny (2 cold starts versus 22), the percentage explodes into the hundreds.
- Note what is actually true on that application: for 11% more memory than ATC'20 spends, we take **1 cold start instead of 2**. We are not worse there; the measurement just cannot express it.

Does this contaminate the headline? No, and it is worth being able to say why. The cohort result sums cold starts and GB-minutes across 1,999 applications before interpolating, so the aggregate curve is smooth and has no steps to jump over. The artefact only appears when scoring a *single* near-deterministic application on its own.

The fix in the demo is to sweep the price on a much denser grid, so the curve is resolved rather than stepped over, and to choose the non-win examples from applications where the method genuinely does not help rather than from ones where the measurement breaks down.

One thing to watch for in the live demo

- The demo's per-application table shows each policy at **one** setting of its own knob, so the rows **do not spend the same memory**. On some apps our window will look worse there.
- That is not a loss, it is a different spend. There is a warning row on screen saying so, and a "match the spend" control. The real claims are the cohort-wide matched-cost ones.

- If someone points at a row where we look worse, say exactly this: *"those rows are at different memory spends; here is the matched-cost comparison."*

---

12. The bug that nearly got into the report, and why it matters

Tell this story if asked "what did you get wrong". It is the strongest integrity point available.

- While drawing a figure, I labelled each window with its cost under our own cost formula. Our "optimal" window showed a **higher** cost than the baseline's. That is impossible if the optimiser works.
- The bug: a smoothing setting was adding 241 fake data points to apps that had only about 992 real ones — a 24% fictional prior claiming gaps were spread evenly out to four hours. It was quietly making the optimiser stupid, choosing a 9-minute window where the true optimum was 21.
- **The unit tests had not caught it**, because they checked the optimiser against a brute-force search of the *same flawed formula*. Both sides shared the bug and agreed perfectly. What caught it was plotting the result against a quantity that had no reason to agree.
- **Now the uncomfortable part.** The broken version scored **better on the test data** (74.3% vs 69.3%). Keeping it would have been easy and would have handed us a bigger headline.
- So we tuned that setting on the **validation** days instead — the data we never report on. On validation, the broken setting ranked **dead last of seven**.
- Meaning: choosing on the test score would have picked the worst-generalising setting in the grid and attached our biggest number to it. There is now a unit test that pins that setting to the validation record so it cannot drift back.

---

13. Every number, and what it means

- **+78.8%** — derived window vs the deployed Azure policy, at matched memory cost, 1,999 applications, held-out days. *For the same memory bill, users hit roughly a fifth as many cold starts.*
- **+67.6%** — the same, against a baseline strengthened to tune both of its percentiles over 36 configurations. *The number to quote if challenged on fairness.*
- **−427.5%** — the donor-history control. *The proof that the method needs each app's own past, not just sensible window sizes.*
- **+26.3%** — conditioning alone, with a random-key control at −1.4%. *A real, separate, smaller contribution, and a strict generalisation of the deployed policy.*
- **+68.6%** — both changes together. *Worse than B alone; they compete for the same data.*
- **88.95%** — share of the next idle gap that is irreducible. *Nobody will ever "solve" cold starts. We are competing over the remaining 11%, and that is why the honest framing matters.*
- **11.0%** — share explained by the previous gap, the most of any context. *This chose our conditioning variable.*
- **+0.99** — correlation between how much information an app's gaps carry and how much we gained on it. *The mechanism is what we claim it is.*
- **26.8% to 76%** — the range across memory budgets. *Where the method matters and where it stops mattering.*

---

14. Honest weaknesses — say these before you are asked

- **It is a simulator, not a deployment.** The trace gives per-minute invocation counts and no cold-start timings, so cold starts are counted in a minute-tick simulator using a published latency model. We did not deploy to a real platform.
- **One warm instance per application.** We model aggregate concurrency, as the ATC'20 and FaasCache evaluations do. Real platforms may hold several instances.
- **One trace.** Azure Functions 2019. We have not shown this transfers to another provider; carrying it to the Huawei trace is Project-II work.
- **Minute granularity.** That is all the data has. Sub-minute effects are invisible to us.
- **The gain fades at loose memory budgets**, as covered above.
- **Change A and Change B do not compose**, and we do not fully understand how to get both. We can describe the conflict but not resolve it.

---

15. Likely questions, with answers

- **"Is 78.8% at matched cost, or did you just keep more containers warm?"** Matched cost. Every policy is swept over its own cost knob and compared by interpolating at a common memory budget. We learned this the hard way — an earlier version showed 38% that vanished entirely once we matched the budget.
- **"Your policy has two free parameters, the baseline has one."** It did, so we removed the asymmetry: 36 baseline configurations, best frontier taken, chosen with the test period in view. We still win by 67.6%, positive at every budget.
- **"How do you know you haven't overfitted to the test days?"** Temporal splits, never shuffled — train 1-8, validate 9-10, test 11-14. All statistics built online from gaps that have already closed, with a unit test asserting that changing the future cannot change a past decision. The one hyper-parameter was chosen on validation, and the report shows that the test-flattering setting ranks last there.
- **"Why not deep learning?"** Because we measured where the signal is before choosing a mechanism. It is one coarse discrete variable. An exact-match lookup gets thousands of samples per cell; a learned embedding with eight neighbours gets eight. We built the learned version first — it lost to its own control.
- **"Is this just the ATC'20 policy with extra steps?"** For Change A, deliberately yes: their policy is recovered exactly as our limiting case, which is what makes it a strict generalisation. Change B is a different thing entirely — it replaces the percentile rule with an optimal- stopping decision, and turns the tuning knob into a price.
- **"What is novel here?"** Three things. One: nobody had separated *which histogram you look at* from *how you turn a histogram into a window*; both had been treated as one policy. Two: nobody had measured how much of a serverless idle gap is predictable at all, and that measurement selects the method rather than justifying it afterwards. Three: the warm window had never been derived from a cost model — it was always a percentile.
- **"What did you get wrong?"** Tell the smoothing-bug story from section 12. Then: we predicted our renewal-state features would help most beyond four hours idle; the gain is actually at one to four hours. The prediction failed as stated and is reported that way.

---

16. A five-minute presentation path

1. **The problem** (20s) — scale to zero means cold starts; the decision is how long to hold an idle container; too short costs latency, too long costs money.
2. **The baseline** (20s) — Azure's deployed policy: one histogram per app, two percentiles read off it. That is the whole thing.
3. **The measurement** (60s) — before building anything we measured what is even predictable. 89% is irreducible. That killed our first three ideas and chose the method. Show `information_audit.png`.
4. **Change B, the headline** (90s) — the window was never derived, only guessed. Show `derived_why.png`: the percentile spans 60 minutes of empty distribution; the cost model stops where holding stops paying. Then `derived_frontier.png`: +67.6% against the best of 36 tuned baselines, positive at every budget, and here is where it runs out.
5. **Why believe it** (60s) — `derived_composition.png`. The donor control at −427%. Then the non-composition finding.
6. **Change A** (30s) — the second, smaller result: +26.3% from conditioning, control at −1.4%, deployed policy recovered as the limiting case.
7. **Close** (30s) — the validation story from section 12, and the negative results. "We report the controls, including the ones that contradicted us."

If you only get one sentence: *Azure's deployed policy reads its warm window off two fixed percentiles of an idle-time histogram; we derive it instead by minimising what the window actually costs, which cuts cold starts by 78.8% at matched memory spend — and the same optimiser given another application's history is five times worse than the baseline, so the gain is genuinely each system's own past preparing it for its own future.*